

Course Project

CS 348 - Fall 2026

Project Description

Important Dates (all deadlines 11:59pm)

- Milestone 0 (preparation): Fri, Oct 2 [Not Graded, but important]
- Milestone 1 (proposal): Fri, Oct 23 [30%]
- Milestone 2 (progress report): Fri, Nov 13 [30%]
- Milestone 3 (demo, code, and report): Thu, Dec 3 [40%]
- Assessment-type switching cutoff: Fri, Oct 30

You have until Oct 30 to switch between the assignment- or project-based grading schemes. After that, the teams in the project-based grading **cannot** switch to the assignment-based assessment. *Individuals dropping out of the project-based grading must receive consent from the other team members (cc-ed in an email to the IA, ISC, and the instructor); if the team size reduces to below 4 afterwards, the entire team will be switched to the exam-based grading.*

Overview

The goal of this project is to build a database-driven application from the ground up. Your application will have a set of "basic features", together with one required feature on concurrency and correctness. No prior experience in developing such applications is assumed. This document also describes a number of possible ideas. You will be working in a group of 4-5 students – minimum of 4 and a maximum of 5 students. There is the possibility to perform consultations with TAs/IAs regarding your idea and development of the project throughout the term.

Submission and Grading

There will be three milestones; see Important Dates above for due dates. You will find the details of what to submit for each checkpoint later in this document under respective sections. Only one member of the group needs to submit the deliverables by the deadline on Learn – “CS 348 Fall 2026” at Submit > Dropbox on behalf of the entire project team; make sure all members’ names are added to the submission. Because of the open-ended nature of course projects, certain instructions may not apply to your particular project; when in doubt, consult the TAs/IAs.

One TA will be assigned to each group, for the purpose of grading and helping direct the projects. The project constitutes **a maximum of 25%** towards your final marks in the course.

Milestone 0 ensures that each student finds a team and gets familiar with the environment setup. Milestone 1’s proposal helps you get a feel for the amount of work involved and if it meets the minimum requirements of the depth and scope for a course project while ensuring that you are planning a doable project. Detailed grading rubrics can be found on later pages.

Late submission policy: For each milestone, you have 2 extra days after the deadline for submissions and the entire team loses 25% worth of marks for each additional day, i.e., we subtract that many marks from your actual score for that particular milestone.

Enrolling in Groups

You will indicate the groups you are with by enrolling into a group on Learn by Milestone 0’s deadline. To enroll in a group on Learn, please follow the steps described in this documentation: <https://uwaterloo.atlassian.net/wiki/spaces/ISTKB/pages/1548779704/Groups+-+Students>.

Choose one group number and stick to this group until the end of the project and milestones cycle. Students should make all their submissions for each milestone through the same group link. One student uploads the files on behalf of the entire group.

Teamwork

The project should be completed in 4 or 5-person teams. Regardless of the team size, an equal amount of work is expected and the same grading scale will be applied. All team members will receive the same score for the project, with one exception: D7, the individual question period at the demo, is scored per student. However, if certain team members are not contributing equally to the project, the instructor will consult the relevant TA and will allocate scores proportional to the contribution. You are required to report each team member's effort and progress in the milestone and final reports. **If there is any problem working with the team members that you cannot resolve by yourself, bring it to the instructor's attention as soon as possible.** Last-minute complaints of the form "my partner did nothing" will not be entertained. You can use Piazza to pitch your project ideas by presenting a few lines of your ideas (project domains or types you are interested in) and the number of additional team members you are looking for.

Platform Issues

To develop the project, you must use MySQL on local machines. We provide some sample codes and instructions for you to start with one of two options. If you wish, you may use other languages, tools, or application development frameworks. Setting up the whole application/database stack is non-trivial and will be a rewarding experience. However, the course staff can only support the technologies used in the provided examples.

Tasks Overview

The goal of this project is to build a database-driven application. Specifically, you will need to complete the following tasks over the course of this semester. Note that different members of a team can work on some of these tasks concurrently.

1. Pick your favorite data management application that uses relational data. It should be relatively substantial, but not too enormous. Several project ideas are described at the end of this document, but you are encouraged to come up with your own. When picking an application, keep the following questions in mind:
 - a. How do you plan to acquire the data to populate your database? Use of real datasets is highly recommended. You may use program-generated "synthetic" datasets if real ones are too difficult to obtain.
 - b. How are you going to use the data? What kind of queries do you want to ask? How is the data updated? Your application should support both *queries* (i.e., reading data) and *updates* (i.e., writing data).
2. Design the database schema. Start with an E/R diagram and convert it to a relational schema. Identify any constraints that hold in your application domain, and code them as database constraints. If you plan to work with real datasets, it is important to go over some samples of real data to validate your design (in fact, you should start Task 7 below as early as possible, in parallel to Tasks 3-6). Do not forget to apply database design theory and check for redundancies.
3. Create a sample database using a small dataset. You may generate this small dataset by hand. You will find this sample database very useful in testing because large datasets make debugging difficult.

It is a good idea to write some scripts to create/load/destroy the sample database automatically; they will save you lots of typing when debugging.

4. Design a user interface for your application (e.g. command line or web-based). Think about how a typical user would interact with the database. Do not spend too much time refining the look of your interface; you just need to understand the basic “flow” in order to figure out what database operations are needed in each step of the user interaction.
5. Write SQL queries that will supply dynamic contents for the interface you designed for Task 4. Also, write SQL code that modifies the database on behalf of the user. You may hard-code the query and update the parameters. Test these SQL statements in the sample database.
6. Choose an appropriate platform for your application. JDBC? Python or PHP? JavaScript or plain HTML? Start by implementing a “hello world” type of simple database-driven application, deploy it in your development environment, and make sure that all parts are working together correctly. The course website will provide pointers to working examples.
7. Acquire the large “production” dataset, either by downloading it from a real data source or by generating it using a program. Make sure the dataset fits your schema. For real datasets, you might need to write programs/scripts to transform them into a form that is appropriate for loading into a database. For program-generated datasets, make sure they contain enough interesting “links” across rows of different tables, or else all your join queries may return empty results. Keep in mind that the school servers have limited capacity.
8. Test the SQL statements you developed for Task 5 in the large database. Do you run into any performance problems? Try creating some additional indexes to improve performance. Try creating too many indexes and see how the performance differs.
9. Implement and debug the application and the user interface. Test your application with the smaller sample database first. You may need to iterate the design and implementation several times in order to correct any unforeseen problems.
10. Test your application with the production dataset. Resolve any performance problems.
11. Make your application fancier! You may add additional features such as well designed/user-friendly interface; efficient query answering; support for multi-user access control; handle concurrent users; well-tested applications (corner cases, potential security issues).

Milestone 0: Project Preparation

You should have formed a team and started thinking about tasks 1-6. In particular, your team should get familiar with one of the platforms and be able to run “hello world” type of simple database-drive applications (task 6). An example (ProjectNote-PHP-MySQL.pdf) is provided on Learn under Course Project to help you start. The goal of milestone 0 is to ensure that you have found a team, and your team has already tried out the chosen programming platform and environment (as switching platforms later will be painful and time-consuming).

Submit the following electronically under “Project Milestone 0” on Learn (Dropbox) **Fri, Oct 2 11:59pm**.

1. A progress report **report.pdf** should at least contain:

- A brief description (1-2 paragraphs) of your application: which dataset do you plan to use, who are the users of the application, who are the administrators of the database systems etc, what functionalities you may like to support.
- A brief description (1-2 paragraphs) of your choice of platform (school servers, local machines) and user interface (e.g., how will users interact with your application? command line or web or GUI, etc?)
- A section on **members**, that lists the members of your team, and for **each member**, a description of effort and progress made by this member to date.
- A link to the **GitHub repo** you will use throughout the project to store your source code. We recommend having 4 directories at the top: milestone-0, milestone-1, milestone-2, milestone-3. For each milestone, place the required documents and data under the corresponding folder. When needed use a separate README file for each directory. Share the Github repo with your TA using their Github username. If you want to organize your repo differently, this is fine, as long as you explain your organization clearly in the README of the repo.

2. The source code directory in the **GitHub repo** should at least contain:

- README.md to describe how to create and load your sample database to your chosen platform. You don’t have to use the datasets described in the report. Toy datasets (e.g. a single table) can be used.
- Source code for your “hello world” type of simple database-drive applications with your sample database (e.g. the application allows user to connect to the database, to select some rows from a table, etc.). This just ensures you have (installed and) tested the database system and simple application code on school servers or your own machines for your interested applications.

Grading Rubrics for Milestones 1,2, and 3

We will use the rubric in Table 1 for grading milestones 1, 2, and 3.

R1-R14 will be graded based on the **report.pdf** submitted for each milestone. Note that not every milestone needs to have completed each item we list in R1-R14. See Table 2 to see which items are required for each milestone.

- **R1. Application high-level description:** A high-level description (1-2 paragraphs) of your application: which dataset do you plan to use, who are the users of the application, who are the administrators of the database systems etc, and what functionalities you may like to support.
- **R2. System support description:** A description (1-2 paragraphs) of your choice of programming framework chosen for the application (e.g. locally deployed or on school servers or on the cloud).
- **R3. Database with sample dataset:** A description of the sample data, how the data is generated/cleaned/imported for your application, and how to use the data to populate your database.
- **R4. Database with production dataset:** A description of the production data, how the data is generated/cleaned/imported for your application, and how to use the data to populate your database.
- **R5. Schema design**
 - R5a. Assumptions: A list of assumptions that you are making about the data being modelled. For example, “user id is unique”.
 - R5b. An E/R diagram for your database design: Full points for a well-thought E/R diagram to realize the application and assumptions described in the report. -1 point for any unclear explanation of terms.
 - R5c. Relational data model: A list of database tables with keys declared (relational data model). Full points for proper translation of your E/R diagram into a relational data model. -1 point for any incorrect translation.
 - R5d. Design decisions and revisions: A short log (roughly half a page) recording the reasoning behind your schema. It must contain:
 - * **Alternatives considered.** For at least two non-obvious modelling choices, describe the alternative(s) you rejected and why (e.g. three tables vs. four, a multi-valued attribute vs. a separate relation, storing a derived value vs. computing it, creating a view vs. not). State the trade-off in terms of redundancy, query cost, or which constraints you were able to enforce.
 - * **At least one revision forced by real data.** Describe a case where the production data violated an assumption you listed in R5a, or did not fit the schema you designed, and what you changed in response. Include the evidence: the rows, counts, or query output that revealed the problem.

This item is graded on the quality of the reasoning, not on whether your first design was correct. A team whose original schema survived contact with the real data unchanged should say so, and show the query they ran to confirm it.
- **R6. Basic Feature/Functionality 1:** We expect at least 5 distinct basic features for your applications (R6-R10). We consider that different features require different underlying query templates. For example, ‘SELECT a FROM r WHERE b = 0’ is considered the same as ‘SELECT b FROM r WHERE a < 1’.

- R-a. Feature interface design: A description of the interface of this feature. Who are the users of this feature? How will a user interact with your application and how does the application respond to the users' requests? You can write a brief English description of how users interact with the interface (e.g., “the user selects a car model from a pull-down menu, clicks on the ‘go’ button, and a new page will display all cars of this model that are available for sale”. This is an example of one Basic Feature/Functionality).
- R-b. SQL query, testing with sample data: We expect you to have written down the related SQL queries in the report for this feature and tested them on the sample database. Please include the related SQL queries and their expected output in the report. You must submit your test database records in some format (e.g., csv), and a test-sample.sql and test-sample.out files that respectively show what the SQL query is and what the query's output is on the test database.
- R-c. SQL query, performance measurement on production data: You must submit your “production” database (your intended real database) records in some format (e.g., csv), and test-production.sql and test-production.out files that respectively show what the SQL query is and what the query's output is on the production database.

For this item we require a **measurement**, not a description. For **each** feature, report:

- * the output of EXPLAIN (or EXPLAIN ANALYZE) for the query, run against the **production** database; and
- * the **total statement time** on the production database. Every team measures this the same way: run the query in a mysql session, then read the time back from the performance schema's events_statements_history table in that same session. Warm up with one run, then report the **median of five**. The exact statements to use are given in the Appendix, together with the machine you measured on.

For at least **two** of your five features, additionally show a **before/after** pair: run the query, apply one change intended to make it faster (an index, a rewrite, a different join order), re-run it, and report both query plans and both timings. Then explain *why* the plan changed — which access path the optimizer switched to, and why that is cheaper for your data. A complete worked example — query, plan, index, and the plan again — is given in the Appendix on page 18.

A description of tuning you *might* do, with no plans and no numbers, earns at most “Needs work” for this item. A change that turned out **not** to help receives full credit, provided you report the plans and timings that show it did not: a negative result, honestly measured, is a correct answer and a well learned lesson.

- R-d. Implementation, snapshot, testing: We expect a fully implemented and tested feature. The report can include the feature snapshot, the test case description (how to test the interface of this feature).
- **R7-R10. Basic Feature/Functionality(s) 2-5** is the same as R6.
 - **R11. Concurrency and correctness:** A database is used by more than one person at a time; this item asks you to show what that does to your application, and what you did about it.
 - R11a. *Exhibit the problem.* Find an operation in your application where two users acting at the same time produce a wrong result. These are almost always read-modify-writes: two requests booking the last seat, two people updating the same balance, two users claiming the same username. Reproduce it with two mysql sessions against your production database, and give: the statements in the order you ran them, what each session saw, the final state of the row, and what the correct result should have been. Name the anomaly — for example *lost update* or *write skew*.

- R11b. *Make it correct.* Change your application or your schema so that the same interleaving no longer produces the wrong result, and show it: the same two sessions, the same order, the right answer. How you do this is your decision, and there is more than one defensible choice (e.g., transactions, integrity constraints, CHECKs). State what you changed and why you expected it to work.
- R11c. *Measure what it costs.* Correctness is rarely free. Run the same two-session interleaving with and without your fix, and report how long each session took in both cases. Say what you observe (e.g., locking behavior), and what you would expect to happen if many users hit this operation at once.

A single SELECT run twice will not show anything interesting here. The operations that break are the ones that read a value, decide something on the basis of it, and then write.

Teams that wish to go further — other anomaly types, explicit isolation levels, deadlock detection, SKIP LOCKED queue processing, or any other concurrency work you find interesting — are welcome to write it up under this item; it is not required.

- **R12. Member:** A description on **members**, that list the members of your team, and for **each member**, a description of effort and progress made by this member to date.
- **R13. Tool and AI use:** A short appendix — half a page is plenty — describing how your team used AI assistants or code-generation tools on this milestone, if you used them at all. State what you used them for (e.g. drafting the E/R diagram, writing the loader script, generating test data), one specific thing such a tool produced that was **wrong or unsuitable**, and what you changed to fix it. A team that did not use these tools should say so in one line and will receive full marks.

This appendix is not used to penalise tool use. These are ordinary tools and you are expected to use them. It is graded on the specificity of your account: “we used AI for some parts” earns no marks, whereas “we asked for a normalized schema and got a 3NF design that silently dropped the constraint that two bookings cannot overlap for the same room, which we added back as a trigger” earns full marks.

- **R14. Adversarial testing of your SQL:** Showing that a query returns the right answer on data you chose is weak evidence: you chose the data. This item asks for stronger evidence. Pick **two** of your five basic features and, for each, give in the report:
 - the query, as you finally have it;
 - a **plausible but incorrect variant** of it — the query a reasonable person, or a code-generation tool, might well have written instead;
 - a small database instance (20 rows or fewer is plenty) on which the two queries return **different** results, together with both outputs; and
 - one or two sentences saying why the plausible query fails on that data.

For at least one of the two, the difference must fall in one of these categories: *NULL handling* (e.g. NOT IN against a subquery containing NULL, COUNT(*) versus COUNT(col), SUM over an empty group), *join multiplicity* (e.g. INNER versus LEFT JOIN where a row has no match, or duplicate rows introduced by a one-to-many join), or *boundaries and ordering* (e.g. ties under ORDER BY ... LIMIT, or a condition placed in WHERE when it belongs in HAVING).

If either query came from a code-generation tool, say so — this is the same disclosure R13 asks for, and it is not held against you. The point of the item is the *data*: a team that can construct the instance on which two similar-looking queries disagree has understood what its own query means. A team that cannot has not yet checked.

C1-C5 will be graded based on the source code posted on your GitHub repo. For milestones 0-2, we will not test your code, but the code will be scanned by the TA. For Milestone 3, you will do a live demo followed by a testing session, during which the TAs will test your applications. Make sure what you describe in the report reflects the code (e.g. test-sample.sql, test-sample.out, application platform). The source code directory should at least contain:

- **C1. README file:** A README file describing how to create and load your sample database; and how to run your working database-driven application, and what feature it currently supports.
- **C2. SQL code (create tables, etc):** Files containing the SQL code used for creating tables, constraints, stored procedures and triggers (if any).
- **C3. SQL queries for features, test-sample.sql, test-sample.out** A file test-sample.sql (or other corresponding files) containing the SQL statements, and a file test-sample.out showing the results of running test-sample.sql over your sample database. For the final milestone, also include test-edge.sql and test-edge.out for the R14 instances, so that we can re-run them.
- **C4. SQL queries for features, test-production.sql, test-production.out** A file test-production.sql (or other corresponding files) containing the SQL statements, and a file test-production.out showing the results of running test-production.sql over your production database.
- **C5. Application code:** The end-to-end application code that implements all the claimed features for the corresponding milestone.
- If applicable, any code for downloading/scraping/transforming real data that you have written.

D1-D7 will be graded based on the **demo**. Your presentation should clearly illustrate the following items for your application.

- D1. Application overview
- D2. How to use your application? (demonstrating functionalities/features)
- D3. System support for your application and briefly explain the back-end queries for each feature
- D4. The contribution of each member
- D5. Summary

In addition, you will be marked on the overall quality of the presentation and demo (i.e., how polished is the demo presentation). This constitutes D6.

D7. Individual question period. D7 is the one item on this rubric scored **per student rather than per team**. After the presentation, the TA will pick a member of your team at random, choose one of the SQL queries behind your features, and ask that member to explain it: what it returns, why it is written the way it is, which tables it touches, and why those tables exist in your schema. The TA will repeat this with other members and other queries, and may follow up on any decision you recorded under R5d.

What is being assessed is whether you understand the system your team submitted. You may use any tools you like while building the project; you may not bring them into the question period. A team whose members cannot explain their own schema and queries will lose these marks even if every artifact they submitted is correct.

Task No.	Items	No attempt	Needs work	Acceptable	Excellent	Score
1	R1. Application high-level description	0	0.5	1.0	1.0	
6	R2. System support description	0	0.5	1.0	1.0	
3	R3. Database with sample dataset	0	1.0	1.5	2.0	
7	R4. Database with production dataset	0	1.5	2.5	3.0	
2	R5. Schema design					
	R5a. Assumptions	0	1	1.5	2.0	
	R5b. E/R diagram	0	1.5	2.5	3.0	
	R5c. Relational data model	0	1.5	2.5	3.0	
	R5d. Design decisions and revisions	0	2	3	4.0	
	R6. Basic Feature/Functionality 1					
4	R-a. Interface design	0	0.25	0.5	0.5	
5	R-b. SQL query, testing with sample data	0	0.25	0.75	1.0	
8	R-c. SQL query, plan and timing on production data	0	0.5	1.5	2.0	
9/10	R-d. Implementation, snapshot, testing	0	0.5	1.0	1.5	
	R7. Basic Feature/Functionality 2					
4	R-a.	0	0.25	0.5	0.5	
5	R-b.	0	0.25	0.75	1.0	
8	R-c.	0	0.5	1.5	2.0	
9/10	R-d.	0	0.5	1.0	1.5	
	R8. Basic Feature/Functionality 3					
4	R-a.	0	0.25	0.5	0.5	
5	R-b.	0	0.25	0.75	1.0	
8	R-c.	0	0.5	1.5	2.0	
9/10	R-d.	0	0.5	1.0	1.5	
	R9. Basic Feature/Functionality 4					
4	R-a.	0	0.25	0.5	0.5	
5	R-b.	0	0.25	0.75	1.0	
8	R-c.	0	0.5	1.5	2.0	
9/10	R-d.	0	0.5	1.0	1.5	
	R10. Basic Feature/Functionality 5					
4	R-a.	0	0.25	0.5	0.5	
5	R-b.	0	0.25	0.75	1.0	
8	R-c.	0	0.5	1.5	2.0	
9/10	R-d.	0	0.5	1.0	1.5	
11	R11. Concurrency and correctness					
	R11a. Exhibit the anomaly	0	1.5	2.5	3.0	
	R11b. Make it correct	0	1	2	2.5	
	R11c. Measure what it costs	0	1	1.5	2.0	
	R12. Member	0	1	1.5	2	
	R13. Tool and AI use	0	1	1.5	2	
	R14. Adversarial testing of your SQL	0	1	2	2.5	
	C1. ReadMe	0	0.5	1.0	1.5	
	C2. SQL code (create tables, etc)	0	1	1.5	2	
	C3. SQL queries for features, test-sample.sql, test-sample.out	0	0.5	1.0	1.5	
	C4. SQL queries for features, test-production.sql, test-production.out	0	1	1.5	2	
	C5. Application code	0	4	5.5	7	
	D1. Demo-app overview	0	0.5	1.0	1.5	
	D2. Demo-use app	0	2	4	5.5	
	D3. Demo-system support	0	2	4	5.5	
	D4. Demo-user contr.	0	0.5	1.0	1.5	
	D5. Demo-summary	0	0.5	1.0	1.5	
	D6. Demo presentation	0	1	2	2.5	
	D7. Individual question period (per student)	0	4	7	10	

Table 1: Grading Rubrics for Milestones 1, 2, and 3

Items	Milestone 1	Milestone 2	Milestone 3	Max Point
R1. Application high-level description	*	*	*	1.0
R2. System support description	*	*	*	1.0
R3. Database with sample dataset	*	*	*	2.0
R4. Database with production dataset	-	*	*	3.0
R5. Schema design				
R5a. Assumptions	*	*	*	2.0
R5b. E/R diagram	*	*	*	3.0
R5c. Relational data model	*	*	*	3.0
R5d. Design decisions and revisions	-	*	*	4.0
R6. Basic Feature/Functionality 1				
R-a. Interface design	*	*	*	0.5
R-b. SQL query, testing with sample data	*	*	*	1.0
R-c. SQL query, plan and timing on production data	-	*	*	2.0
R-d. Implementation, snapshot, testing	-	*	*	1.5
R7. Basic Feature/Functionality 2				
R-a.	*	*	*	0.5
R-b.	*	*	*	1.0
R-c.	-	*	*	2.0
R-d.	-	*	*	1.5
R8. Basic Feature/Functionality 3				
R-a.	*	*	*	0.5
R-b.	*	*	*	1.0
R-c.	-	*	*	2.0
R-d.	-	*	*	1.5
R9. Basic Feature/Functionality 4				
R-a.	*	*	*	0.5
R-b.	*	*	*	1.0
R-c.	-	*	*	2.0
R-d.	-	-	*	1.5
R10. Basic Feature/Functionality 5				
R-a.	-	*	*	0.5
R-b.	-	*	*	1.0
R-c.	-	*	*	2.0
R-d.	-	-	*	1.5
R11. Concurrency and correctness				
R11a. Exhibit the anomaly	-	-	*	3.0
R11b. Make it correct	-	-	*	2.5
R11c. Measure what it costs	-	-	*	2.0
R12. Member	*	*	*	2
R13. Tool and AI use	*	*	*	2
R14. Adversarial testing of your SQL	-	-	*	2.5
C1. ReadMe	*	*	*	1.5
C2. SQL code (create tables, etc)	*	*	*	2
C3. SQL queries for features, test-sample.sql, test-sample.out	*	*	*	1.5
C4. SQL queries for features, test-production.sql, test-production.out	-	*	*	2
C5. Application code	*	*	*	7
D1. Demo-app overview	-	-	*	1.5
D2. Demo-use app	-	-	*	5.5
D3. Demo-system support	-	-	*	5.5
D4. Demo-user contr.	-	-	*	1.5
D5. Demo-summary	-	-	*	1.5
D6. Demo-presentation-quality	-	-	*	2.5
D7. Individual question period (per student)	-	-	*	10
Maximum scores per milestone	34	59	100	

Table 2: Basic Requirements for Milestones 1, 2, and 3; * means “required”, and - means “not required”

How to compute the overall score of your project? Your final score will be a weighted average of your score for each milestone as follows:

- Milestones 1, 2, and 3 comprise respectively 30%, 30%, and 40% of your final score.
- If you obtain s_1 , s_2 , and s_3 for milestones 1, 2, and 3. Then your overall project score will be

$$\frac{s_1}{34} * 30 + \frac{s_2}{59} * 30 + \frac{s_3}{100} * 40 \tag{1}$$

Milestone 1: Project Proposal

You should have completed Tasks 1-6 and have started thinking about 7. If you plan to work with real data, you should also have made significant progress on Task 7 (you should at least ensure that it is feasible to obtain the real dataset, transform it, and load it into your database). **Submit** the following electronically under “Project Milestone 1” on Learn - Dropbox by **Fri, Oct 23 11:59pm**:

1. A progress report **report.pdf** should at least contain:

- R1. A high-level description (1-2 paragraphs) of your application: which dataset do you plan to use, who are the users of the application, who are the administrators of the database systems etc, what functionalities you may like to support.
- R2. Describe the system support (1-2 paragraphs) (Task 6): e.g. programming framework chosen for the application (e.g. OS that runs for your application and DBMS or school server), interface for connecting with the db, etc.
- R3. A plan for getting the sample (toy) data to populate your database, as well as some sample data (Task 3).
- R5. Design database schema (Task 2)
 - A list of assumptions that you are making about the data being modelled.
 - An E/R diagram for your database design.
 - A list of database tables with keys declared (relational data model).
- R6a, R6b, R7a, R7b, R8a, R8b, R9a, R9b. We expect at least **4 features’ description, their corresponding query template, sampled queries, and expected output**.
- R12. A section on **members**, that list the members of your team, and for **each member**, a description of effort and progress made by this member to date.
- A link to the GitHub repo.

2. The source code directory should at least contain:

- C1. A README file describing how to create and load your sample database; and how to run your working database-driven application, and what feature it currently supports.
- C2. Files containing the SQL code used for creating tables, constraints, stored procedures and triggers (if any).
- C3. A file test-sample.sql (or other corresponding files) containing the SQL statements you wrote for Task 5, and a file test-sample.out showing the results of running test-sample.sql over your sample database (not the full dataset).
- C5. Code implementing a simple but working database-driven application on your chosen platform (1-2 simple features/functionalities can be run), which can serve as a starting point for completing your project.

Only the deliverables necessary for Milestone 1 will be graded in this milestone. If a group has achieved more, that will be graded in the future milestones.

Milestone 2: Project Progress Report

You should have completed Tasks 1-9 and have made good progress on 10. **Submit** the following electronically under “Project Milestone 2” on Learn Dropbox by **Fri, Nov 13, 11:59pm**:

1. Add details to all the requirements from milestone 1 with more details. Highlight (e.g. [use a different color](#)) the changes and new descriptions you made if there are (e.g. new changes, E/R diagram, and a list of tables). If the TA thinks that the report.pdf from milestone 1 has a lot of details already, you can keep the same details and get full points. Examples for details: you can add a snapshot of the features, elaborate the features, or add new features.

A progress report **report.pdf** should at least contain:

- R1. A high-level description (1-2 paragraphs) of your application.
- R2. Describe the system support (1-2 paragraphs) (Task 6).
- R3. A plan for getting the sample (toy) data to populate your database (Task 3).
- R4. A plan for getting the production data to populate your database (Task 7).
- R5. Design database schema (Task 2).
- R6-R10. A description of your application features. Try to list all the **key** features/functionalities of the application. Feel free to list additional features that your group is interested in implementing if possible. We expect all [5 features’ description, their corresponding query template, sampled queries, and expected output](#), and at least [3 features implemented and tested](#).
- R12. A section on **members**.
- A link to the GitHub repo.

2. The source code directory should at least contain:

- C1. A README file describing
 - How to create and load your sample database; and how to run your working database-driven application, and what feature it currently supports.
 - [How to generate the “production” dataset and load it into your database](#). Do not submit the production dataset itself through if it is too big; instead, submit the URL where you download/scrape the raw data (if applicable), and the code that extracts and transforms (or generates) the production dataset. State which features have been implemented and state which files contain the implementation. We expect at least 3 features.
- C2. Files containing the SQL code used for creating tables, constraints, stored procedures and triggers (if any) for sample database and [production database](#).
- C3. A file test-sample.sql (or other corresponding files) containing the SQL statements you wrote for Task 5 and a file test-sample.out showing the results of running test-sample.sql over your sample database (not the full dataset).
- C4. A file [test-production.sql containing the SQL statements you wrote for Task 8](#). You may wish to modify some queries to return only the top 10 result rows instead of all result rows (there might be lots for large datasets) and a file [test-production.out showing the results of running test-production.sql over the production dataset](#).
- C5. Code implementing a simple but working database-driven application on your chosen platform. [This code includes at least 2 more features than the version in milestone 1](#).

Milestone 3: Demo, Code, and Report

At the end of the semester, between Nov 30 (Mon) - Dec 3 (Thu) you will need to present an **in person** demo of your system to your TA followed by a testing session where the TA will test your features (ultimately testing the correctness of the queries that underlie the features of your application).

Instructions for demo:

- By Fri Nov 27: Email your assigned TA to find a day/time for 20 minute in person time. **[You lose 2 marks if fail to do so.]**
- **Note:** To ensure a smooth demo presentation, the team should **rehearse** before the demo session. Feel free to use slides, video, or any tools that enable a smooth presentation. We will not grade the tools or demo formats, but the overall quality of the contents and clarity of the demo constitutes a small part of your marks (D6).
- Every member should be present and contribute to the demo.
- Your demo should be maximum 10 minutes. The remaining 10 minutes will be for live Q&A.

The demo presentation will be graded based on the following items.

- Presentation contents. Your presentation should clearly illustrate the following items for your application.
 - D1. Application overview
 - D2. How to use your application? (demonstrating functionalities/features)
 - D3. System support for your application and briefly explain the back-end queries for each feature
 - D4. The contribution of each member
 - D5. Summary
 - D6. Overall quality clarify of the presentation. Try to make your demo as engaging and compelling as you can.

During the testing session, the TAs can take off some marks of your Milestone 3 mark for the features that are not working properly. The demo/report/code should include all the items listed in Table 1.

After your demo presentation, complete your report by Dec 3rd. Submit your finalized report.pdf (including the GitHub repo) electronically under "Project Final" on Learn Dropbox before **Dec 3rd, 11:59pm**.

Please **highlight** the changes made by you in report.pdf since the last milestone (M2). The grading of the final submission will be mainly based on your demo and the final report.

Project Ideas

Below is a list of possible project ideas for which high-quality datasets exist. Of course, you are welcome to come up with your own.

Entertainment, sports, or financial websites

Examples include those that allow visitors to explore information about movies, music, sports, games, stocks, etc. There are already many commercial offerings for such purposes. While there is less room for innovation, there are plenty of examples of what a good website would look like, as well as high-quality, well-formatted datasets. For example, IMDb makes their movie database available <http://www.imdb.com/interfaces>; historical stock quote can be downloaded and scraped from many sites such as Yahoo! and Google Finance. This project is well-suited for those who just want to learn how to build database-backed websites as beginners. You can always spice things up by adding features that you wish those websites had (e.g., different ways for summarizing, exploring, and visualizing the data).

Websites providing access to datasets of public interest

If you are interested in doing some good to society while learning databases, this project is for you. There are many interesting datasets “available” to the public, but better ways of accessing and analyzing them are still sorely needed. Here are some examples:

- Open Canada (<https://search.open.canada.ca/data/>) has a huge compilation of data sets produced by the Canadian government. My research group has done some work in this space. You can look in the introduction sections of these works to get a better feel for the overall goals and motivation of the Canadian government for publishing these datasets: [1]<https://www.jeffjianshao.com/papers/governor.pdf>, [2]<https://www.youtube.com/watch?v=7c0SifFA5v4>, [3]<https://openproceedings.org/2024/conf/edbt/paper-44.pdf>.
- The Supreme Court Database (<http://scdb.wustl.edu/data.php>) tracks all cases decided by the US Supreme Court.
- US government spending data (<https://www.usaspending.gov/>) has information about and database downloads of government contracts and awards.
- Federal Election Commission (<https://www.fec.gov/data/>) has campaign finance data to download as well as nice interfaces for exploring the data.
- GovTrack.us (<http://www.govtrack.us/developers>) tracks all bills through the Congress and all votes casted by its members. The Washington Post has a nice (albeit outdated) website (<http://projects.washingtonpost.com/congress/113/>) for exploring this type of data (in predefined ways), but you can be creative with additional and/or more flexible exploration and analysis options. Vote Smart (<https://votesmart.org/>) has a wealth of additional, useful information on votes, such as issue tags, synopses and highlights.
- Each state legislature maintains its own voting records. For example, you can find North Carolina’s here: <http://www.ncleg.net/Legislation/voteHistory/voteHistory.html>. Some states provide records in already structured formats, but for others, you may need to scrape their websites.
- The Washington Post maintains a list of datasets (<http://www.washingtonpost.com/wp-srv/metro/data/datapost.html>) that have been used to generate investigative news pieces. Most of these datasets hide behind some interface and may need to be scraped. Use this list for examples of what datasets are “interesting” and how to present data to the public effectively.

- Stanford Journalism Program maintains a list of curated transportation-related datasets (<http://www.datadrivenstanford.org/>).
- National Institute for Computer-Assisted Reporting maintains a list of datasets of public interest (<http://www.ire.org/nicar/database-library/>). Use this list for examples of what datasets are “interesting”—they are generally not available to the public, but there may be alternative ways to obtain them.

Your task would be to take one of such datasets, design a good relational schema, clean up/restructure the data, and build an application for the public to explore the dataset. You are welcome to come up with your own.

Appendix: a worked example for R-c

R-c asks you to *measure* a query rather than describe what you might do to it. This appendix shows what that looks like from start to finish. Everything printed below is real output from MySQL 8.0.46 — yours will differ in the numbers, but not in shape.

The setup

Suppose your production database has a `person` table holding 200,000 rows:

```
CREATE TABLE person (  
  person_id INT NOT NULL,  
  name VARCHAR(120) NOT NULL,  
  born SMALLINT,  
  PRIMARY KEY (person_id)  
);
```

and one of your features looks a person up by name:

```
SELECT * FROM person WHERE name = 'Lena N. Tan';
```

Step 1: get the plan before you change anything

```
mysql> EXPLAIN SELECT * FROM person WHERE name = 'Lena N. Tan'\G  
id: 1  
select_type: SIMPLE  
table: person  
partitions: NULL  
type: ALL  
possible_keys: NULL  
key: NULL  
key_len: NULL  
ref: NULL  
rows: 201082  
filtered: 10.00  
Extra: Using where
```

(Ending the statement with `\G` instead of a semicolon prints one field per line, which is far easier to read than the wide table that a plain semicolon gives you.)

Three fields carry almost all of the information here:

- `type: ALL` — a **full table scan**. This is the thing you are usually trying to get rid of.
- `key: NULL` — no index was used, because there is none on `name`.
- `rows: 201082` — the optimizer expects to examine about 201,000 rows in order to answer a query that returns 13.

`EXPLAIN` only reports the *plan*. `EXPLAIN ANALYZE` actually *runs* the query and reports what really happened, including time:

```
mysql> EXPLAIN ANALYZE SELECT * FROM person WHERE name = 'Lena N. Tan'\G  
-> Filter: (person.`name` = 'Lena N. Tan')  
      (cost=20244 rows=20108) (actual time=1.09..52.3 rows=13 loops=1)  
-> Table scan on person  
      (cost=20244 rows=201082) (actual time=0.0675..37 rows=200000 loops=1)
```

Read `actual time=1.09..52.3` as “the first row came back after 1.09ms and the last after 52.3ms”. Note the contrast between `rows=201082` (the estimate) and `actual ... rows=200000` (what was really scanned) to find 13 matching rows. The optimizer expects to examine about 201,000 rows in order to answer a query that returns 13. This figure is an *estimate*, not a count. The database derives it by sampling a small number of index pages, so it drifts by a few percent from one `ANALYZE TABLE` to the next and can even come out slightly larger than the number of rows the table actually holds. What matters is not its precision but that the optimizer chooses its plan on the basis of it.

Step 2: make one change

```
mysql> CREATE INDEX idx_person_name ON person(name);
mysql> ANALYZE TABLE person;
```

Run `ANALYZE TABLE` afterwards so the optimizer’s statistics reflect the new index.

Step 3: get the plan again

```
mysql> EXPLAIN SELECT * FROM person WHERE name = 'Lena N. Tan'\G
      id: 1
  select_type: SIMPLE
        table: person
  partitions: NULL
         type: ref
possible_keys: idx_person_name
          key: idx_person_name
        key_len: 482
          ref: const
         rows: 13
   filtered: 100.00
      Extra: NULL

mysql> EXPLAIN ANALYZE SELECT * FROM person WHERE name = 'Lena N. Tan'\G
-> Index lookup on person using idx_person_name (name='Lena N. Tan')
   (cost=4.55 rows=13) (actual time=0.0466..0.0585 rows=13 loops=1)
```

Step 4: say why the plan changed

This is the part that earns the marks. A full-credit answer says something like:

With no index on name, type was `ALL` and key was `NULL`: MySQL read all 200,000 rows of `person` and discarded the ones that did not match. Adding `idx_person_name` changed the plan to `type: ref`, an index lookup that reads only the 13 matching rows — 52.3ms down to 0.0585ms. The price is an extra index to maintain on every write to `person`, worth paying here because the table is read far more often than it is written.

What to submit for each feature

1. The query.
2. `EXPLAIN` (or `EXPLAIN ANALYZE`) output on the **production** database.
3. A timing figure, measured the same way by every team so that the numbers mean something when compared. Run your query in a `mysql` session, then — in that same session — read its time back out of the performance schema:

```
mysql> SELECT ROUND(TIMER_WAIT/1e9, 3) AS ms
-> FROM performance_schema.events_statements_history
-> WHERE SQL_TEXT LIKE 'SELECT * FROM person WHERE name%'
-> ORDER BY EVENT_ID DESC LIMIT 5;

+-----+
| ms    |
+-----+
| 0.223 |
| 0.141 |
| 0.145 |
| 0.236 |
| 0.290 |
+-----+
```

The LIKE pattern just has to match the start of your own query; the one above picks out the running example of this appendix. `TIMER_WAIT` is recorded in picoseconds, so dividing by 10^9 gives milliseconds. Run the query once to warm up, then five more times, and report the **median** of the five — 0.145ms for the run shown above. State the machine you measured on.

This is the *whole statement*: parsing, optimising, executing, and returning the rows. Report this figure for both the “before” and the “after”.

Two practical notes:

- `EXPLAIN ANALYZE` requires MySQL 8.0.18 or later. On an older server you have `EXPLAIN` only, which is acceptable — report timings separately in that case.
- Repeated runs of the same query are usually faster than the first, because the pages are already in the database buffer pool. Run each query once to warm up before you record a time, and use the same warm/cold condition for both the “before” and the “after” measurement, or you are not comparing like with like.

Course Project

CS 348

Supplementary Material II:
PHP & MySQL Sample Application

This sample application works with a local server. This document presents a simple PHP application on local server together with MySQL. PHP (<https://www.php.net/>) is a server scripting language, and a powerful tool for making dynamic and interactive Web pages. MySQL (<https://www.mysql.com/>) is one of the most popular database system used with PHP.

If this setting is chosen for your project, you may consider web as the user interface and in Milestone 0, you should get familiar with the following basics of PHP and MySQL.

- Download and install MySQL and PhP on your local computer.
- Create a database on MySQL and user account.
- Create a PHP based application that can be executed on your local computer. A sample code is provided on Learn (testDB.php)
- Establish a connection from your PHP application to your database.
- Process some SQL Statements with PHP.
- Display query results on the web with PHP.
- Retrieving and modifying values in the database using PHP.

Get Started

Step 1: Download and install MySQL on your own computer. Instructions can be found at <https://dev.mysql.com/doc/refman/8.0/en/installing.html>

After installing MySQL, create a database 'testDB' and a user and password; then grant the user permission to 'testDB'.

```
$ mysql -u root -p (or sudo mysql -u root)
mysql> CREATE DATABASE testDB;
mysql> USE testDB;
mysql> CREATE TABLE student(uid DECIMAL(3, 0) NOT NULL PRIMARY KEY, name
VARCHAR(30), score DECIMAL(3, 2));
mysql> INSERT INTO student VALUES(1, 'alice', 0.1);
mysql> INSERT INTO student VALUES(2, 'bob', 0.4);
mysql> SELECT * FROM student;
```

Now you should have created a database 'testDB' with a single table 'user' consisting of 2 rows.

Note that you are using root as the user. If you want to create different user account other than root account, you may consider create new users. For example, the code below creates a user with username 'user1' with password 'Password0!'.

```
mysql> create user 'user1'@'localhost' identified by 'Password0!';
mysql> grant all on testDB.* to 'user1'@'localhost';
mysql> alter user 'user1'@'localhost' identified with mysql_native_password by
'Password0!';
```

The last line of the code above is the traditional method to authenticate (it is not very secure), but it is a quick way for you to start your PHP application.

Step 2: Download, install and configure PhP on your local computer. <https://www.php.net/docs.php>
For example, if you are using MacOS, you can start PHP as follows

```
$ php -S 127.0.0.1:8000
```

If you store your first PHP file in home directory (test.php), you can type "http://127.0.0.1:8000/test.php" in your browser to run test.php. A sample of test.php code is shown below.

```
<!DOCTYPE html>
<html>
<body>
<?php echo "My first PHP script!  </br>"; ?>
</body>
</html>
```

Step 3: Establish connection from your PHP application to your database instance and display query results on the browser with PHP.

If both PHP and MySQL are ready, you can run a testDB.php (download it from Learn) to test the connection using PHP application to your database instance (by typing “http://127.0.0.1:8000/testDB.php” in your browser.) You should see the message “2 students in the student table” shown on your browser if there are two students in your ‘student’ table in ‘testDB’ database.

```
<!DOCTYPE html>
<html>
<body>
<?php
$servername = "127.0.0.1";
$username = "user1";
$password = "Password0!";
$dbname = "testDB";
// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
die("Connection failed: " . $conn->connect_error); }
$sql = "SELECT uid FROM student";
$result = $conn->query($sql);
echo $result->num_rows." students in the student table";
$conn->close();
?>
</body>
</html>
```

Step 4: Get fancier!

The sample code shows a toy example to start the project. The data management application should be relatively substantial, but not too enormous. Several project ideas are described at the end of the main project document, but you are encouraged to come up with your own. As web will be considered as the user interface for this application, the project team needs to ensure the interface support a rich set of interactions between users and the database (e.g. enter a value, delete an entry, etc.) The interface can cover a rich set of queries and also handles exceptions (e.g. invalid input, etc.)

You are free to choose other applications other than PHP, for the user interface.